SIMATS ENGINEERING

ASSESSMENT TOOL 2 — CONSTRAINT-BASED PROBLEM SOLVING

Course: Artificial Intelligence | Course Code: CSA17

Note: Question 2 in the supplied question paper does not show the exact locations of S, G, or obstacles. To demonstrate the method, a sample 5×5 grid is assumed below. Replace it with the given class grid if your faculty has provided one.

QUESTION 1 — Hospital Doctor Scheduling

Given constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3 (D2's shift must be earlier than D3's).
3. Only one doctor can work each shift.
4. D3 cannot work the Morning shift.
5. Each doctor must receive exactly one shift.

Backtracking Search

Step 1: Assign D2 = Morning. This is allowed because no constraint prevents D2 from working Morning.

Step 2: Assign D3. D3 cannot work Morning, and D2 must be before D3. Therefore, D3 can be Afternoon or Night. Try D3 = Afternoon.

Step 3: The only remaining shift is Night, so assign D1 = Night. Check constraint 1: D1 cannot work Night. This fails, so backtrack.

Step 4: Keep D2 = Morning and try D3 = Night. This satisfies D2 before D3 and D3 not Morning.

Step 5: Assign the remaining Afternoon shift to D1. Check all constraints:

Constraint	Check	Result
D1 cannot work Night	D1 = Afternoon	Satisfied
D2 before D3	Morning is before Night	Satisfied
One doctor per shift	One doctor in each shift	Satisfied
D3 cannot work Morning	D3 = Night	Satisfied
Every doctor gets one shift	D1, D2, D3 each assigned once	Satisfied

Final valid schedule: Morning — D2; Afternoon — D1; Night — D3.

This schedule satisfies all five constraints. Backtracking was used when the first trial (D3 = Afternoon) led to an invalid assignment for D1.

QUESTION 2 — Robot Path Finding in a 5×5 Grid

Assumed grid

Coordinates are written as (row, column). S = (1,1), G = (5,5). Obstacles are at (2,2), (2,4), (3,2), and (4,4). The robot can move Up, Down, Left, or Right; each move costs 1.

S (1,1)	.	.	.	.
.	X	.	X	.
.	X	.	.	.
.	.	.	X	.
.	.	.	.	G (5,5)

Algorithm note: Breadth-First Search (BFS) expands nodes level by level and uses a FIFO queue. Since every move has equal cost (1), BFS finds an optimal shortest path. Manhattan distance is h = |row – 5| + |column – 5|. A heuristic priority queue is normally associated with Greedy Best-First Search or A*, not standard BFS; therefore, the steps below show the BFS queue and also report Manhattan distance for the nodes on the chosen path.

BFS step-by-step node expansion

Use neighbor order: Right, Down, Left, Up. A node is added only if it is inside the grid, not an obstacle, and not visited before.

Step	Expanded node	New valid nodes added to queue	Queue after expansion (front → back)
1	(1,1) S	(1,2), (2,1)	(1,2), (2,1)
2	(1,2)	(1,3)	(2,1), (1,3)
3	(2,1)	(3,1)	(1,3), (3,1)
4	(1,3)	(1,4), (2,3)	(3,1), (1,4), (2,3)
5	(3,1)	(4,1)	(1,4), (2,3), (4,1)
6	(1,4)	(1,5)	(2,3), (4,1), (1,5)
7	(2,3)	(3,3)	(4,1), (1,5), (3,3)
8	(4,1)	(4,2), (5,1)	(1,5), (3,3), (4,2), (5,1)
9	(1,5)	(2,5)	(3,3), (4,2), (5,1), (2,5)
10	(3,3)	(3,4)	(4,2), (5,1), (2,5), (3,4)
11	(4,2)	(4,3), (5,2)	(5,1), (2,5), (3,4), (4,3), (5,2)
12	(5,1)	—	(2,5), (3,4), (4,3), (5,2)
13	(2,5)	(3,5)	(3,4), (4,3), (5,2), (3,5)
14	(3,4)	—	(4,3), (5,2), (3,5)
15	(4,3)	—	(5,2), (3,5)
16	(5,2)	(5,3)	(3,5), (5,3)
17	(3,5)	(4,5)	(5,3), (4,5)
18	(5,3)	(5,4)	(4,5), (5,4)
19	(4,5)	G = (5,5)	(5,4), (5,5)
20	(5,4)	—	(5,5)
21	(5,5) G	Goal reached	Stop

One optimal path

(1,1) → (1,2) → (1,3) → (1,4) → (1,5) → (2,5) → (3,5) → (4,5) → (5,5)

Number of moves = 8. Since each move costs 1, total path cost = 8 × 1 = **8**.

Manhattan distance values along this path

h(1,1)=8, h(1,2)=7, h(1,3)=6, h(1,4)=5, h(1,5)=4, h(2,5)=3, h(3,5)=2, h(4,5)=1, h(5,5)=0.

Conclusion: BFS finds a shortest valid route because all moves have equal cost and obstacles are excluded. The optimal path cost in this assumed grid is 8.

QUESTION 3 — Autonomous Rescue Robot

(a) Analyze the constraints and their effect

1. **Four-direction movement:** The robot may move only Up, Down, Left, or Right. Diagonal moves are not allowed, so it must plan a route through neighboring cells.
2. **Obstacles:** Blocked cells cannot be entered. The robot must detect or infer blocked routes and choose another path.
3. **Limited sensing:** It can observe only adjacent cells, so it does not know the whole map at the start. It must gather information as it moves.
4. **Movement cost:** Every move costs 1.
5. **Risky zones:** Entering a risky zone adds 2 to the normal cost, so entering it costs 1 + 2 = 3. The robot should avoid risky zones when a safer, lower-cost route exists.
6. **Dynamic knowledge:** The robot updates its map after observing neighboring cells. Its plan may change when it discovers an obstacle or a risk.

(b) Suitable solution approach: Uniform Cost Search with online replanning

Step 1: Start at S. Put S in a priority queue with cumulative cost g(S)=0.
Step 2: Sense the four adjacent cells and mark each as free, blocked, or risky if this information is available.
Step 3: For each reachable neighbor, calculate the new path cost: add 1 for a normal cell, or 3 for entering a risky zone. Never add blocked cells.
Step 4: Add or update each candidate in the priority queue, ordered by lowest cumulative cost g(n). Store the parent of each cell to reconstruct the path.
Step 5: Remove the lowest-cost cell from the queue and expand it. If it is the goal G, stop; the path found is minimum-cost when the relevant map/costs are known.
Step 6: In the partially observable setting, move toward the best currently known next cell, sense again, update the map, and replan if a newly discovered obstacle or risk changes the route.
Step 7: Continue until G is reached. Reconstruct the route by following parent links and report the total movement and risk cost.

Cost example: A normal move costs 1. A move into a risky zone costs 1 + 2 = 3. If a route has 5 normal moves and 2 risky-zone entries, total cost = (5 × 1) + (2 × 3) = 11.

(c) AI concepts

• **Intelligent agent:** The robot senses its surroundings, processes the information, chooses an action, and moves toward the survivor.
• **Rationality:** A rational robot chooses actions that best achieve its goal while minimizing total cost and avoiding blocked or unsafe cells, based on the information available at that time.
• **Environment type:** The environment is partially observable, sequential, dynamic/unknown as the robot discovers new information, and discrete (grid cells and four possible actions). It is also a single-agent environment. If risk information is uncertain, it can be considered stochastic; if outcomes are fixed, it is deterministic.

(d) Why this approach is suitable

Uniform Cost Search is suitable because normal and risky moves have different costs. It expands the lowest cumulative-cost option first, rather than merely choosing the route with the fewest steps. Online sensing and replanning are necessary because the robot cannot see the complete building map initially. Together, these methods support low-cost navigation while responding to newly discovered obstacles and risks.

Final summary

Q1: D2 — Morning, D1 — Afternoon, D3 — Night.
Q2: For the clearly stated sample grid, shortest path cost = 8.
Q3: Use Uniform Cost Search with online sensing and replanning to minimize total cost.